

TP Chapitre 2 :

Pipelines agiles, versionnement et orchestration avec dlt, dbt et Dagster

Demandé par : Pr. Aitdaoud Mohammed

Réalisé par : Aldiebes ghanem Israa

Master : IA

Groupe : 1

1. Introduction et objectifs

Ce travail pratique vise à construire une chaîne de traitement de données moderne permettant d'automatiser les différentes étapes de manipulation des données. L'accent est mis sur l'utilisation d'outils favorisant l'organisation, la maintenabilité et la reproductibilité des processus de données.

Le pipeline développé assure le passage des données depuis une source CSV jusqu'à leur transformation et leur orchestration selon le flux suivant :

CSV local → dlt → DuckDB → dbt → Dagster

Technologies utilisées

- **dlt** : extraction et chargement des données dans la base de stockage.
- **DuckDB** : base de données analytique légère utilisée pour stocker les données.
- **dbt** : préparation, transformation et modélisation des données.
- **Dagster** : planification et orchestration des différentes tâches du pipeline.
- **Git** : suivi des versions et collaboration sur le projet.
- **venv, pip et requirements.txt** : gestion de l'environnement de développement et des dépendances.

2. Vue d'ensemble du pipeline

Le diagramme suivant présente les principaux composants du pipeline ainsi que leurs interactions. Il met en évidence le parcours des données, depuis leur ingestion à partir d'un fichier CSV jusqu'à leur traitement et leur orchestration, tout en intégrant les mécanismes de gestion de versions et d'automatisation du déploiement.

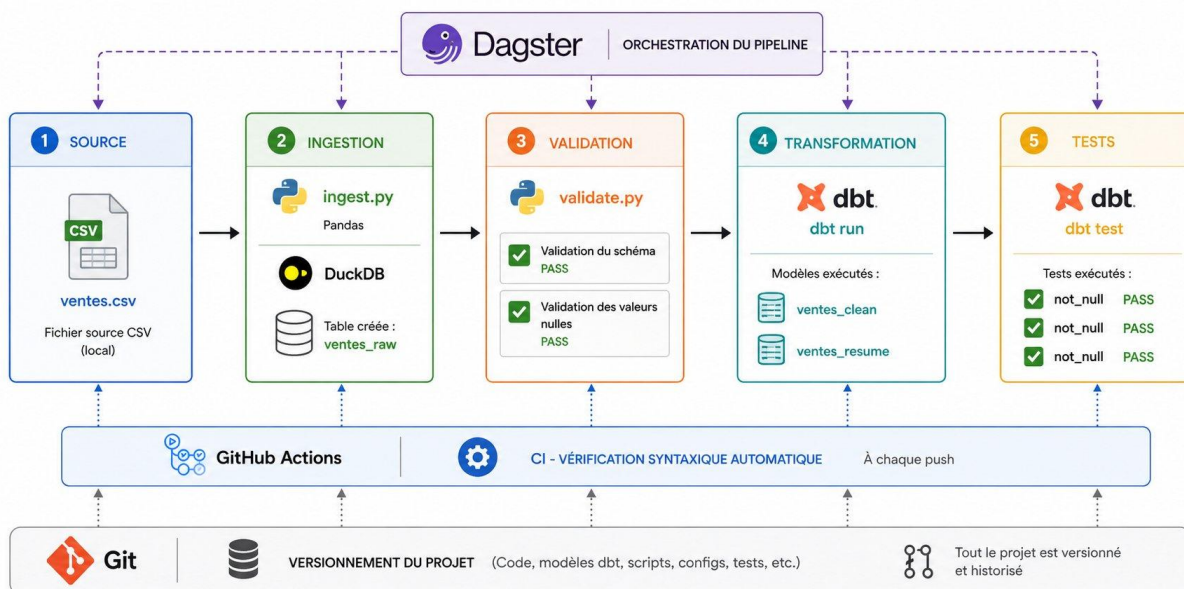


Figure 1 — Schéma complet du pipeline MLOps (CSV → DuckDB → dbt → Dagster → CI)

3. Mise en place de l'environnement

Structure du projet

L'arborescence du projet a été organisée de façon modulaire :

tp_chapitre2_pipeline/ data/ventes.csv pipeline/ingest.py, validate.py, orchestrate.py
dbt_pipeline/models/, dbt_project.yml, profiles.yml .github/workflows/ci.yml requirements.txt, .gitignore

Environnement virtuel

Afin de garantir l'isolation de l'environnement de travail et la reproductibilité du projet, un environnement virtuel Python a été mis en place. L'ensemble des bibliothèques nécessaires au fonctionnement du pipeline a été répertorié dans un fichier **requirements.txt**, généré à l'aide de la commande **pip freeze**.

Activation de l'environnement (Windows PowerShell) :

```
.venv\Scripts\Activate.ps1
```

Principales bibliothèques utilisées :

- dlt
- dagster
- dagster-webserver
- dbt-core
- dbt-duckdb
- pandas
- duckdb
- pyyaml

4. Etapes du pipeline

Etape 1 — Ingestion

Le script ingest.py lit le fichier ventes.csv avec pandas et charge les données dans DuckDB en créant la table ventes_raw.

```
(.venv) PS D:\pfm\ML\tp_chapitre2_pipeline> python  
Ingestion terminée : ventes_raw créée dans DuckDB
```

Figure 2 — Résultat de l'exécution de ingest.py

Etape 2 — Validation minimale

Le script validate.py vérifie que toutes les colonnes requises sont présentes et qu'aucune valeur nulle n'existe dans les colonnes critiques (produit, quantité, prix_unitaire).

```
Validation réussie : schéma et qualité minimale OK
```

Figure 3 — Résultat de l'exécution de validate.py

Etape 3 — Transformation avec dbt

Deux modèles dbt ont été créés :

- ventes_clean.sql : filtre les lignes valides et calcule le chiffre d'affaires (quantite x prix_unitaire)
- ventes_resume.sql : agrégation par catégorie avec total quantité et total CA

```
(.venv) PS D:\pfmML\tp_chapitre2_pipeline\dbt_pipeline> dbt run --profiles-dir .
23:47:46 Running with dbt=1.11.11
23:47:47 Registered adapter: duckdb=1.10.1
23:47:48 Found 2 models, 3 data tests, 485 macros
23:47:48 Concurrency: 1 threads (target='dev')
23:47:48 1 of 2 START sql table model main.ventes_clean ..... [RUN]
23:47:49 1 of 2 OK created sql table model main.ventes_clean ..... [OK in 0.23s]
23:47:49 2 of 2 START sql table model main.ventes_resume ..... [RUN]
23:47:49 2 of 2 OK created sql table model main.ventes_resume ..... [OK in 0.23s]
23:47:49 Finished running 2 table models in 0 hours 0 minutes and 0.93 seconds (0.93s).
23:47:49 Completed successfully
23:47:49 Done. PASS=2 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=2
(.venv) PS D:\pfmML\tp_chapitre2_pipeline\dbt_pipeline>
```

Figure 4 — Execution de dbt run (PASS=2 models)

Etape 4 — Tests dbt

Des tests de qualité ont été définis dans schema.yml pour vérifier l'absence de valeurs nulles sur les colonnes clés. Les 3 tests ont passé avec succès.

```
(.venv) PS D:\pfmML\tp_chapitre2_pipeline\dbt_pipeline> dbt test --profiles-dir .
23:48:32 Running with dbt=1.11.11
23:48:33 Registered adapter: duckdb=1.10.1
23:48:36 Found 2 models, 3 data tests, 485 macros
23:48:36 Concurrency: 1 threads (target='dev')
23:48:36 1 of 3 START test not_null_ventes_clean_produit ..... [RUN]
23:48:36 1 of 3 PASS not_null_ventes_clean_produit ..... [PASS in 0.25s]
23:48:36 2 of 3 START test not_null_ventes_clean_quantite ..... [RUN]
23:48:37 2 of 3 PASS not_null_ventes_clean_quantite ..... [PASS in 0.07s]
23:48:37 3 of 3 START test not_null_ventes_resume_categorie ..... [RUN]
23:48:37 3 of 3 PASS not_null_ventes_resume_categorie ..... [PASS in 0.13s]
23:48:37 Finished running 3 data tests in 0 hours 0 minutes and 1.16 seconds (1.16s).
23:48:37 Completed successfully
23:48:37 Done. PASS=3 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=3
(.venv) PS D:\pfmML\tp_chapitre2_pipeline\dbt_pipeline>
```

Figure 5 — Execution de dbt test (PASS=3 tests)

Etape 5 — Orchestration avec Dagster

Le fichier orchestrate.py définit 4 ops Dagster (ingest, validate, transform, test_data) assembles dans un job appelé ventes_pipeline. Dagster gère automatiquement l'ordre d'exécution et la journalisation.

```
2026-05-31 00:51:02 +0100 - dagster-webserver - INFO - Serving dagster-webserver on http://127.0.0.1:3000 in process 17208
```

Figure 6 — Lancement du serveur Dagster

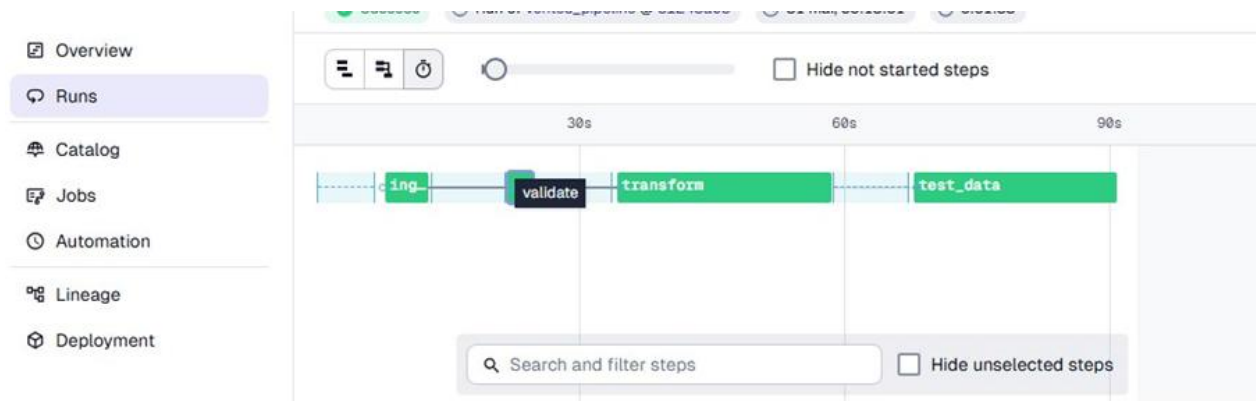


Figure 7 — Interface Dagster : pipeline exécute avec succès (RUN_SUCCESS)

5. Versionnement Git

Chaque étape du pipeline a fait l'objet d'un commit distinct, assurant une traçabilité complète de l'évolution du projet.

```
(.venv) PS D:\pfmML\tp_chapitre2_pipeline> git log --oneline
88ef595 (HEAD -> master) Ajout d'une CI simple
593de5d Ajout de l'orchestration Dagster
5e2d509 Ajout de l'orchestration Dagster
d7161b8 Ajout des modèles dbt et tests
d0a8044 Ajout de l'étape de validation minimale
c58c5b4 Ajout de l'étape d'ingestion
4343144 Ajout arborescence et fichier source ventes.csv
579e39e Initialisation du projet pipeline
```

Figure 8 — Historique Git (7 commits)

6. Intégration continue (CI)

Un workflow GitHub Actions a été configuré dans `.github/workflows/ci.yml`. Il se déclenche à chaque push sur la branche main et effectue les vérifications suivantes :

- Checkout du dépôt
- Installation de Python 3.11
- Installation des dépendances via `requirements.txt`
- Vérification syntaxique des scripts Python avec `py_compile`

7. Réponses aux questions

1. Intérêt d'une architecture en pipeline

Contrairement à un notebook unique où les différentes opérations sont regroupées dans un seul fichier, l'architecture adoptée repose sur une séparation claire des responsabilités. Chaque composant du pipeline est dédié à une tâche spécifique : ingestion des données, validation, transformation et orchestration. Cette organisation améliore la lisibilité du code, facilite la maintenance et permet de tester chaque étape indépendamment. De plus, en cas d'erreur, il devient plus simple d'identifier précisément l'origine du problème.

2. Importance du versionnement avec Git

L'utilisation de Git permet d'assurer le suivi des évolutions du projet au fil du temps. Chaque modification est enregistrée dans l'historique grâce aux commits, offrant ainsi la possibilité de revenir à une version antérieure si nécessaire. Cet outil facilite également le travail collaboratif et garantit une meilleure traçabilité du développement du pipeline.

3. Gestion des dépendances du projet

Le fichier *requirements.txt* centralise l'ensemble des bibliothèques nécessaires au fonctionnement du projet ainsi que leurs versions exactes. Cette approche permet à tout utilisateur ou collaborateur de recréer un environnement identique à l'aide d'une simple commande d'installation, assurant ainsi la reproductibilité des résultats.

4. Utilité de l'étape de validation

L'ajout d'une phase de validation avant les transformations contribue à renforcer la qualité des données. Cette étape permet de vérifier la présence des colonnes attendues ainsi que l'absence d'anomalies critiques, telles que des valeurs manquantes. En cas de problème, le pipeline s'interrompt immédiatement afin d'éviter la propagation d'erreurs dans les étapes suivantes.

5. Contrôle qualité avec dbt

L'outil dbt permet non seulement de transformer les données mais également d'effectuer des contrôles qualité grâce à la commande *dbt test*. Les règles définies dans le fichier *schema.yml* sont appliquées directement aux données transformées afin de vérifier le respect des contraintes établies. Cette vérification complète les contrôles réalisés lors de la phase de validation initiale.

6. Apport de l'orchestration avec Dagster

Dagster automatise l'exécution du pipeline en gérant les dépendances entre les différentes étapes. Il garantit que chaque tâche est exécutée dans le bon ordre et fournit une interface visuelle permettant de suivre l'état d'avancement des traitements. En cas d'échec d'une étape, les suivantes ne sont pas lancées, ce qui contribue à la robustesse globale du processus.

7. Contribution de l'intégration continue

L'intégration continue mise en place via GitHub Actions permet d'automatiser certaines vérifications à chaque modification du projet. Les dépendances sont installées automatiquement et des contrôles de base sont exécutés afin de détecter rapidement d'éventuelles erreurs. Cette démarche contribue à maintenir un niveau minimal de qualité tout au long du développement.

8. Perspectives d'industrialisation

Bien que le pipeline réponde aux objectifs du TP, plusieurs améliorations seraient nécessaires pour une utilisation en environnement de production. Parmi celles-ci figurent la gestion sécurisée des secrets et des variables d'environnement, la mise en place de mécanismes d'alerte en cas d'incident, le développement de tests unitaires plus complets, ainsi que la conteneurisation à l'aide de Docker. L'utilisation de services de stockage cloud, l'intégration d'outils de monitoring et la gestion du cycle de vie des données permettraient également d'améliorer la fiabilité, la scalabilité et la traçabilité de la solution.

8. Conclusion

Ce travail pratique a permis de mettre en œuvre les principaux concepts du DataOps et du MLOps à travers la réalisation d'un pipeline de données complet et reproductible. À partir d'une source de données au format CSV, plusieurs étapes ont été intégrées dans un flux cohérent : ingestion des données dans DuckDB, validation de leur qualité, transformations avec dbt, orchestration via Dagster, gestion des versions avec Git et automatisation des contrôles grâce à GitHub Actions.

La mise en place de cette architecture a mis en évidence les avantages d'une approche modulaire par rapport à une solution centralisée dans un unique notebook. La séparation des responsabilités entre les différents composants facilite la maintenance, améliore la traçabilité des traitements et renforce la reproductibilité du projet. Ces caractéristiques constituent des éléments essentiels pour le développement de pipelines de données robustes et adaptés aux besoins des environnements professionnels.